

Assignment 01

①

Name: Tahsin Raihan Robbani

Id: 22201344

Course: CSE321

Section: 7

The Multilevel Feedback Queue is a dynamic and adaptive CPU scheduling algorithm designed to improve system performance by handling both CPU-bound and I/O bound processes in such way that it ensures high CPU utilization and system responsiveness.

Unfair CPU-Allocation due to I/O-Bound Processes;

MLFQ is designed to adjust the priority of process dynamically based on CPU burst time and I/O wait times. When a process is executed it is assigned to high priority queue and its CPU time is measured. If the process doesn't finish within its time it is moved to ~~and~~ lower priority queue. On the other hand I/O bound processes perform many I/O operations and execute only short burst of CPU instructions which are moved to higher priority queue. But this approach favors I/O bound

process allowing them to receive more CPU time and remain in high priority queue. This imbalance arises because I/O bound processes quickly cycle through CPU and I/O wait states while CPU bound processes remain in the system for longer periods. This can create situation where CPU bound process are starved, receiving less CPU time which undermines the fairness of the system.

Imbalance Leading to Starvation:

The imbalance caused by the dynamic nature of MFCQ scheduling can lead to starvation. This occurs when a CPU bound process is repeatedly moved to a lower queue and cannot access the CPU for long periods due to higher priority I/O process. Starvation undermines the efficiency of the system. It prevents making progress thus violating the principle of fair allocation in multitasking environment. For example, in a scenario where there are many I/O process and a few CPU process. The CPU process may rarely get a chance to execute. As these process keep getting demoted to lower priority queue

(3)

can leading to complete their burst with limit. On the other hand I/O bound process continue to receive access and executed repeatedly. So the CPU bound process has to wait for long time.

Fairness Improving Mechanism :

Several fairness improving mechanism can be introduced to address the unfair allocation of CPU time.

① Aging: Aging is a mechanism in which the priority of a process is gradually increased. The longer it has been waiting in queue. By adjusting the priority of CPU bound process that have been denied for too long aging prevents them from being starved and ensures they eventually receive CPU time. This mechanism is vital for balancing the needs for both I/O-bound and CPU bound processes.

② Priority Boosting: Priority boosting can be used temporarily increase the priority of processes that are stuck in lower priority queue for too long. This ensures that these

4

processes are not left indefinitely waiting for CPU access which leads to improving fairness.

(iii) Time Quantum Adjustment: Time quantum adjust the process based on their characteristics. For example, I/O bound process can be allocated shorter time quantum while CPU bound process require longer quantum to allow them to complete their tasks without frequent interruptions. This would reduce the preemptive switching overhead and ensure that CPU bound tasks are given enough time to progress.

These mechanisms help reduce starvation by ensuring that processes especially CPU bound ones are not unfairly delayed and are eventually given fair share of CPU time.

Quantitative Fairness Evaluation

Fairness in CPU scheduling refers to a equal distribution of CPU time across processes, ensuring that no process is unduly delayed ~~on~~ by others. It becomes especially important in MLFA where processes are classified into different categories with each having distinct requirements. Fairness ensures that each process is allocated CPU time in proportion to its needs, preventing one process from consuming too much CPU time. In MLFA fairness is particularly important because of dynamic nature of priority assignments which can cause significant imbalances if not carefully managed. A fair CPU scheduling algorithm should guarantee that interactive ~~and~~ ~~background~~ processes receive an amount of CPU time depending on their nature that one type consistently dominating the CPU.

Fairness Index

A quantitative fairness metric is essential to measure how well the CPU scheduling algorithm distributes the CPU time among processes. One such metric is Fairness Index which quantifies the fairness of the system:

$$\text{Fairness Index} = \frac{(\sum T_i)^2}{n \times \sum T_i^2}$$

Here, T_i is the CPU time allocated to process i and n is the number of processes in the system.

This formula measures inequality in CPU time distribution.

A higher fairness value indicates more equitable CPU allocation while the lower value indicates disparity.

Let's apply the fairness index to MLFQ. To evaluate fairness in MLFQ using the Fairness Index, consider a workload with mix of I/O and CPU processes. Suppose,

- Process 1 (P1): An I/O bound process, allocated 50 ms of CPU time
- Process 2 (P2): A CPU bound process, allocated 250 ms of CPU time
- Process 3 (P3): An I/O bound process, allocated 30 ms of CPU time.

$$\text{Fairness Index} = \frac{(50+150+30)^2}{3 \times (50^2 + 150^2 + 30^2)} \text{ ms}$$

$$= 0.68 \text{ ms}$$

This index suggest that the CPU time distribution is relatively unfair with the CPU bound process p2. As it is receiving disproportionately more time compared to the I/O bound process.

Effect of System Parameter on fairness:

(i) Increasing the number of queues: It allows more granular priority differentiation, which may improve fairness by better classifying processes.

(ii) Quantum length: Its adjustment influence the speed at which processes are switched. Longer time quantum reduce preemption overhead, benefiting CPU bound processes. while shorter quantum may benefit I/O bound process by allowing the more frequent access to the CPU.

III) Aging Intervals: It also affect fairness by ensuring that long waiting process eventually receive CPU time, Adjusting these intervals can help balance the needs of both interactive and computationally tough tasks.

In conclusion, MLFQ scheduling algorithm is powerful and flexible but still it has fairness issues particularly when it comes to I/O bound process taking CPU time. The imbalance caused by this ~~can~~ can result in starvation for CPU bound process. Introducing aging, priority boosting and time quantum ~~help~~ help reducing the issue. The Fairness Index provides a useful tool for evaluating and quantifying the fairness of CPU scheduling in MLFQ system. By carefully adjusting system parameters, we can optimize MLFQ scheduling to achieve a more balanced and ~~fair~~ fair allocation of CPU time for all processes.